Advanced on-device AI

Offline Memory understands what you type or say with a built-in, rule-based engine that runs entirely on your device. It needs no model download and no internet, and it is always the fallback.

If you want more, you can add an optional on-device LLM. It also runs fully on the device (still no internet), but it is a separate add-on that you install yourself, so the default app stays small and can never be broken by a missing native runtime.

What you need

- A checkout of the app source (this is a developer step, done once).
- A machine set up to build the app: Android Studio / Xcode, Node.js, and the Expo tooling.
- A small GGUF language model file (for example a 1-3B instruct model) that fits comfortably in device memory.

Step 1 - add a native LLM runtime

In your fork, install a React Native LLM runtime and rebuild the native app:

```
npx expo install llama.rn
npx expo run:android   # or: npx expo run:ios
```

Nothing about this step affects a normal build - you are opting in.

Step 2 - implement the engine

Create src/ai/engine/on-device-llm-engine.ts that imports the runtime and implements the AiEngine contract, then registers itself:

```
import { initLlama } from 'llama.rn';
import { registerAiEngine, type AiEngine } from './index';

const engine: AiEngine = {
descriptor: {
id: 'on-device-llm',
label: { bn: '<Bengali label>', en: 'Advanced (on-device LLM)' },
description: {
bn: '<Bengali description>',
en: 'Runs a small model on the device - still offline.',
},
builtIn: false,
},
async isReady() { return modelFileExists(); },
async generate(prompt, opts) { /* run llama.rn here */ },
};

registerAiEngine(engine);
```

Step 3 - register and select it

- Import that module once at startup (for example from app/_layout.tsx) so it registers on launch.
- Ship or download the model file and point isReady() at it.
- Choose the engine in Settings -> AI engine. The selection is stored as the aiEngineId preference.

How the fallback works

resolveActiveEngine() checks your chosen engine first. If it is missing, not registered, or isReady() returns false, it silently returns the built-in rule engine. Free-form features must always keep a non-LLM path, so the app keeps working whether or not the model is present.

Privacy

Both engines run on the device. No prompt, no text, and no model output is ever sent to a server. Adding the LLM does not change that.

Reference: src/ai/engine/README.md and src/ai/engine/types.ts in the app source.